SOFTWARE DESIGNER · AFTERNOON SESSION

# 软件设计师 下午场 · 应用技术题型手册

左栏题型考点 · 右栏典型例题/解题模板 · 案例分析+设计题通关

5道大题 | 150分钟 | 满分75分 | 45分及格 | 含算法/C++/Java选做

目录


01 数据流图DFD

02 数据库设计

03 UML建模

04 算法设计与C语言

05 面向对象程序设计（C++/Java）

06 答题技巧与时间分配


蓝顶卡片 = 题型考点　红顶卡片 = 典型例题/模板

## 01 数据流图 DFD（第1题）

### 1.1 图形符号与分层

考点 CONCEPT

- **外部实体**（矩形/方框）：系统外的数据源或终点
- **加工/处理**（圆形/圆角矩形）：将输入数据流变换为输出数据流
- **数据存储**（两条平行线或开口矩形）：存放数据的文件
- **数据流**（箭头）：数据流动方向，必须有名称
- 分层：顶层图 → 0层图 → 1层图……；**父图子图平衡**（输入输出数据流一致）
- 顶层图只有1个加工，代表整个系统

典型例题

Q 指出图中数据流X的数据源和数据终点分别是什么？

Q 说明父图与子图是否平衡？如何检查？

### 1.2 常见错误与解题套路

考点 CONCEPT

典型例题

- **奇迹**：加工只有输出没有输入
- **灰洞**：输入数据不足以产生输出数据
- 数据流不能从外部实体直接到数据存储（必须经加工）
- 数据流不能从数据存储直接到外部实体（必须经加工）
- 两个实体之间不能直接有数据流（必须经加工）
- 同一加工可以有多个输入/输出，但要有因果关系

典型例题

找出DFD图中存在的错误并说明理由（如：加工P2只有输入无输出）

说明数据流X的组成（数据字典定义）

## 02 数据库设计（第2题）

### 2.1 E-R图转关系模式

考点 CONCEPT

- **实体** → 一张表，属性即列，主键加下划线
- **1:1联系** → 任一端主键作外键加入另一端，或将联系本身并过去
- **1:N联系** → **外键加在N端**（多端加一端主键）
- **M:N联系** → **单独成表**，主键=双方主键组合，联系属性作列
- 弱实体：依赖强实体存在，主键=强实体主键+自身部分键

典型例题

将"读者(读者号,姓名) 借阅(借阅日期,应还日期) 图书(书号,书名)"的E-R图转为关系模式

1:N联系"部门-员工"如何转关系模式?

### 2.2 规范化与SQL

考点 CONCEPT

- 函数依赖、部分依赖、传递依赖判定（同上午场）
- 候选码：能推出所有属性的最小属性组；**从未出现在右边的一定在候选码中**
- 范式提升：拆表消除部分依赖→2NF；消除传递依赖→3NF
- SQL：SELECT...FROM...WHERE...GROUP BY...HAVING...ORDER BY
- WHERE子句**不能用聚合函数**（COUNT/SUM/AVG），HAVING可以

典型例题

已知R(学号,课程号,姓名,课程名,成绩)，F={学号→姓名,课程号→课程名,(学号,课程号)→成绩}，指出存在哪些问题并规范化到3NF

查询选修了"数据库"课程且成绩>80的学生姓名，写出SQL

## 03 UML 建模（第3题）

### 3.1 用例图与类图

考点 CONCEPT

典型例题

- **用例图**：参与者(Actor)用小人，用例用椭圆，系统边界用矩形
- 用例关系：**include（必然包含）**虚线箭头指向被包含；**extend（可选扩展）**箭头指向基础用例；泛化=实线空心三角
- **类图**：类名/属性/方法三栏；"+"public、"-"private、"#"protected
- 类间关系：依赖<关联<聚合<组合<继承（耦合由弱到强）
- 多重性：`1..*` 、 `0..1` 、 `*` 分别表示一对多、可空、任意多

典型例题

Q "预订机票"必然执行"验证身份"，"选座"是可选的。用例图中应使用include还是extend？

Q 识别类图中的"订单"和"订单明细"之间是什么关系？

### 3.2 顺序图、状态图、活动图

考点 CONCEPT

- **顺序图**：横轴=对象生命线，纵轴=时间；消息用实线箭头，返回用虚线箭头；**激活条(长方形)**表示对象正在执行
- **状态图**：状态(圆角矩形)+转换(箭头)+事件/监护条件；初始状态实心圆，终止状态双环
- **活动图**：活动(圆角矩形)、判定(菱形)、泳道(分区表示责任)；同步条(粗横线)表示并发
- 常见考法：补全缺失的消息/状态/转换条件

典型例题

Q 顺序图中，对象A向对象B发送消息"提交订单()"，B处理后返回订单号。如何绘制？

Q 状态图中，"订单"从"待支付"到"已取消"的转换条件应写在什么位置？

## 04 算法设计与 C 语言（第4题）

### 4.1 常见算法策略

考点 CONCEPT

- **分治法**：分解→解决→合并；如归并排序、快速排序、二分查找；递归实现
- **动态规划**：最优子结构+重叠子问题；填表法自底向上；如背包问题、最长公共子序列
- **贪心法**：每步取局部最优；如活动选择、哈夫曼编码、Prim/Kruskal
- **回溯法**：深度优先+剪枝；如N皇后、0-1背包
- **分支限界**：广度优先，适合求最优解
- 时间复杂度：看**最深层循环**执行次数

典型例题

Q 补全二分查找代码中的空：while(low ____ high){mid=(low+high)/2; if(a[mid]==key)return mid; else if(a[mid]>key)high=____; else low=____;}

Q 判断下列算法采用何种策略：①每步选当前最小权值边且不成环 ②大问题分解为独立子问题递归求解 ③保存已求解子问题避免重复计算

### 4.2 C语言填空技巧

考点 CONCEPT

- 先读注释和函数说明，**理解算法意图再填空**
- 关注：循环边界、递归出口、指针移动、返回值

典型例题

Q 斐波那契数列递归代码中，空处应填什么？int fib(int n){ if(n<=1) return n; return _______; }

Q 指出下列冒泡排序代码的时间复杂度，并说明如何提前结束？

- 常见形式：`a[i++]`、`p = p→next`、`return mid`、`break/continue`
- 动态规划题：填数组定义、初始值、状态转移方程、结果取值
- 时间复杂度写大O：O(N)、O(N LOG N)、O($N^2$)、O($2^n$)

## 05 面向对象程序设计 C++/JAVA（第5题）

### 5.1 类、继承与多态

考点 CONCEPT

- C++/JAVA选择题二选一，**选自己熟悉的语言**
- **抽象类**：含纯虚函数(C++)或ABSTRACT方法(JAVA)，不能实例化
- **虚函数/抽象方法**实现多态：父类指针指向子类对象，**运行时动态绑定**
- C++：`virtual`声明虚函数；JAVA：默认动态绑定，STATIC/FINAL/PRIVATE不能重写
- 构造方法：创建对象时自动调用；**子类构造先调父类构造(SUPER/初始化列表)**
- 接口(INTERFACE)：只定义方法签名，可实现多继承；类实现用IMPLEMENTS

典型例题

Q JAVA: 父类ANIMAL有方法SPEAK()，子类DOG重写SPEAK()。ANIMAL A = NEW DOG(); A.SPEAK(); 输出什么?

Q C++: CLASS A{PUBLIC: VIRTUAL VOID F()=0;}; 则F()称为什么？A能否实例化?

### 5.2 填空与改错

考点 CONCEPT

- 常考：补全类定义、构造方法、GETTER/SETTER、接口实现
- JAVA填空：`extends`继承、`implements`实现接口、`super()`调父类构造、`this`当前对象
- C++填空：`:`初始化列表、`virtual`、`new/delete`、`&`引用
- 改错题：方法签名不一致、访问权限错误、缺少返回类型、未实现抽象方法

典型例题

Q JAVA补全：PUBLIC CLASS CIRCLE _____ SHAPE{ PUBLIC _____ DOUBLE AREA(){ RETURN 3.14*R*R; } }

Q C++构造方法填空：CLASS POINT{INT X,Y; PUBLIC: POINT(INT A,INT B){ _____; } };

## 06 答题技巧与时间分配

### 6.1 各题时间与策略

考点 CONCEPT

| 大题 | 建议时间 | 策略 |
| --- | --- | --- |
| DFD | 25分钟 | 符号题送分，先拿稳 |

临场例题

Q 距离考试还有30分钟，DFD和算法大题都没做完，如何取舍?

Q SQL题写出后如何快速自查?

| | | |
|---|---|---|
| 数据库 | 25分钟 | E-R转模式、SQL必考 |
| UML | 25分钟 | 图形符号要准确 |
| 算法 | 35分钟 | 先读注释理解意图 |
| OOP | 25分钟 | 选熟悉语言，细心 |
| 检查 | 15分钟 | 回头查计算与漏题 |

- **每空按点给分**，不会也写合理内容，别留空
- 算法题不懂就写"分治/贪心"等策略名称+大致思路

---

### 备考建议

1. 下午场重理解与应用，建议用近3年真题实战演练，严格计时。
2. DFD/数据库/UML是送分题，务必做到符号准确、格式规范。
3. 算法题先理解注释意图，填空从最确定的开始（出口、边界）。
4. C++/Java任选其一，不必都准备，选自己写代码更顺手的。
5. 祝你顺利通过软件设计师考试！